Auteur : [LUCAS Benjamin]
Version : [1.3]
Date : [14/03/2026]

Version 1.0.0

Documentation et rapport du projet MDD

1. Présentation générale du projet

1.1 Objectifs du projet

Le projet MDD consiste à réaliser une application web full-stack de type réseau de développeurs permettant à un utilisateur de s’inscrire, se connecter, s’abonner à des thèmes, publier des articles et interagir via des commentaires.

Dans le contexte de l’entreprise, l’objectif est de disposer d'un site maintenable, sécurisé, et

facilement évolutif. Pour ce faire, spring boot et angular sont des choix pertinents. La valeur ajoutée attendue : offrir aux utilisateurs une plateforme claire pour suivre les sujets qui les intéressent, et fournir à l'équipe technique une base évolutive reposant sur des conventions de développement lisibles, des DTO dédiés, une sécurité JWT et une couverture de tests.

1.2 Périmètre fonctionnel

Fonctionnalités	Description	Statut
Création d'un compte utilisateur	Formulaire d'inscription, validation des champs, normalisation de l'email et création du compte avec retour d'un JWT.	Terminée
Authentification	Connexion via email/mot de passe, génération d'un token JWT, guards front-end et interceptor HTTP.	Terminée
Consultation des articles	Liste des articles triable (asc/desc), affichage synthétique puis détail d'un article.	Terminée
Publication d'un article	Création d'un article avec rattachement à un thème depuis un formulaire.	Terminée
Commentaires	Ajout de commentaires sur un article.	Terminée
Abonnement aux thèmes	Souscription et désabonnement à des thèmes pour personnaliser le parcours utilisateur.	Terminée
Gestion du profil	Consultation et mise à jour du profil utilisateur connecté.	Terminée
Fonctionnalités avancées	Pagination, barre de recherche, notifications, modération, ajout de thèmes.	À venir

2. Architecture et conception technique

2.1 Schéma global de l'architecture

L'architecture retenue suit une séparation front/back classique. Le front-end Angular gère l'interface, les formulaires, la navigation et l'injection du JWT dans les requêtes HTTP. L'API Spring Boot expose les endpoints REST, applique les règles métier et sécurise les accès. La persistance des données est assurée par MySQL via Spring Data JPA/Hibernate. La documentation technique est exposée avec Swagger/OpenAPI et la qualité est suivie par Jest, Cypress, JUnit et JaCoCo.

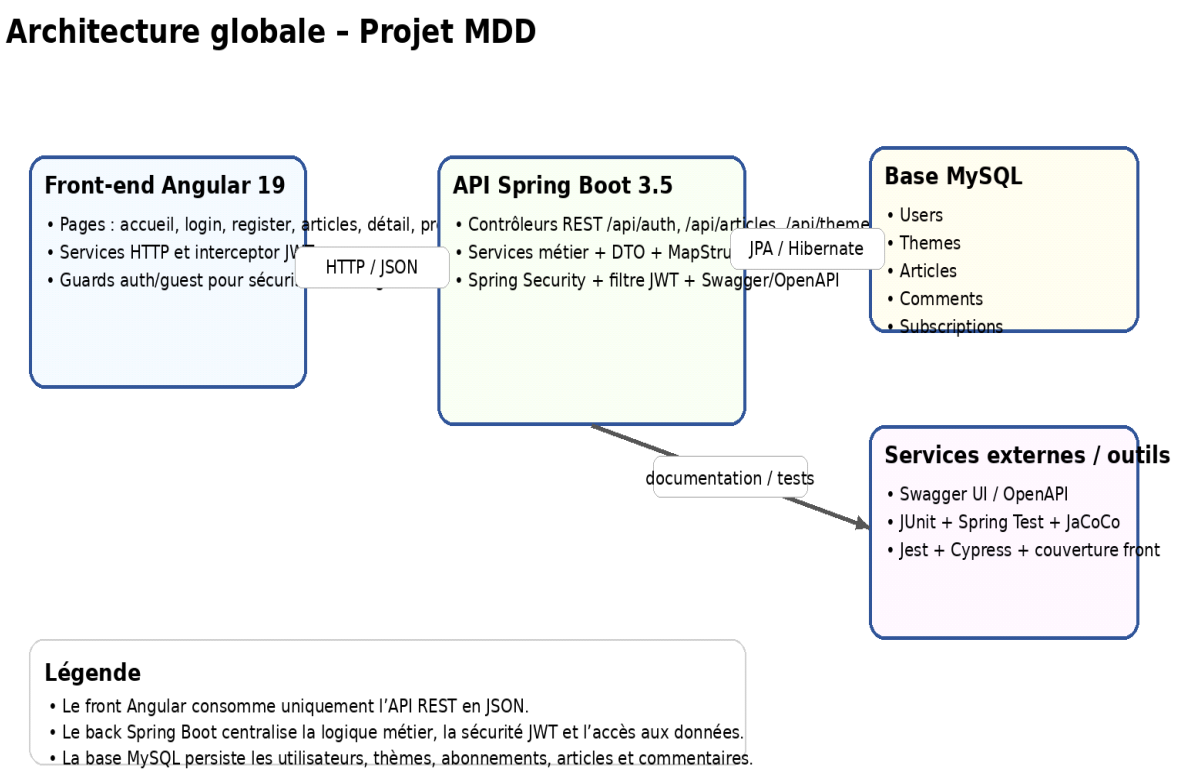


Figure 1 – Architecture globale du projet MDD

Organisation technique :
- front/src/app/pages : écrans métier (authentification, articles, thèmes, profil)
- front/src/app/services : accès API
- front/src/app/core : interceptor JWT
- back/controller : exposition REST
- back/service : logique métier
- back/repository : accès base
- back/models + dto + mapper : persistance, contrats d'échange et transformation

2.2 Choix techniques

Éléments choisis	Type	Lien documentation	Objectif du choix	Justification
Angular 19 + java 21	Framework front-end	https://angular.dev https://docs.oracle.com/en/java/javase/21/	Structuration de l'application.	Respect des standards du parcours.
Angular Material	Bibliothèque UI	https://material.angular.dev	Accélérer la réalisation d'une interface homogène.	Les composants Material (cards, inputs, buttons, select, divider, icon) sont déjà intégrés dans plusieurs pages.

Éléments choisis	Type	Lien documentation	Objectif du choix	Justification
Spring Boot 3.5	Framework back-end	https://spring.io/projects/spring-boot	Structurer rapidement une API REST robuste.	Permet de centraliser configuration, injection de dépendances, validation, tests et intégration web.
Spring Security + JWT	Sécurité	https://spring.io/projects/spring-security	Protéger les routes privées et authentifier les utilisateurs.	Les endpoints publics sont limités à /api/auth/** et la chaîne de filtres applique un contrôle stateless par token.
MySQL + JPA / Hibernate	Base de données / ORM	https://spring.io/projects/spring-data-jpa	Persister les entités métier avec un mapping objet-relationnel.	Le modèle User/Theme/Article/Comment/Subscription est relié par annotations JPA et exploité via des repositories dédiés.
Swagger / OpenAPI	Documentation API	https://springdoc.org	Documenter et tester l'API pendant le développement.	Une configuration OpenAPI est présente avec schéma bearerAuth pour les endpoints sécurisés.
Jest + Cypress + JUnit + JaCoCo	Tests et qualité	https://jestjs.io | https://www.cypress.io | https://www.jacoco.org/jacoco/	Sécuriser les régressions et mesurer la couverture.	Le dépôt contient des tests unitaires front, des scénarios E2E Cypress et des rapports JaCoCo / coverage HTML générés.

2.3 API et schémas de données

L'API est organisée par ressource métier : authentification, articles, thèmes et espace personnel. Les contrôleurs exposent des endpoints REST simples, les DTO encadrent les entrées/sorties et la validation Jakarta sécurise les formats attendus. Deux niveaux de représentation sont utilisés pour les articles : un DTO léger pour les listes et un DTO détaillé pour la fiche article avec commentaires.

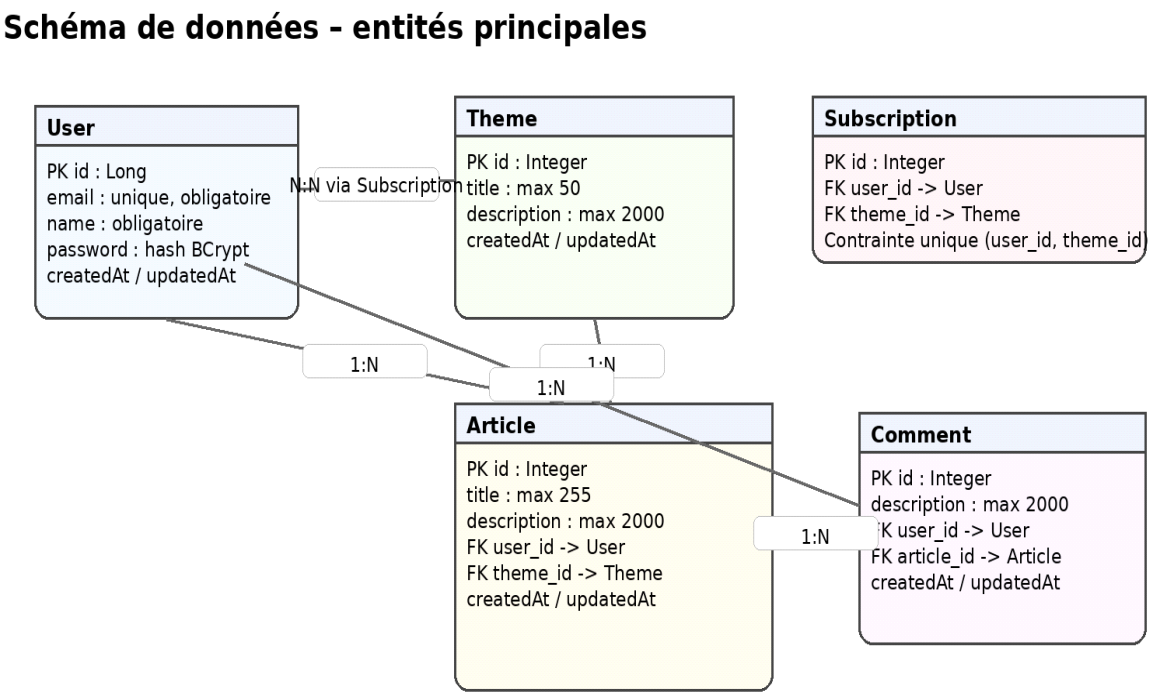


Figure 2 – Schéma simplifié des entités et relations

Exemples JSON :
- POST /api/auth/login
  Requête : {"email":"user@example.com","password":"Password1!"}
  Réponse : {"token":"<jwt>"}

- POST /api/articles
  Requête : {"themeId":1,"title":"Angular 19","description":"Retour d'expérience..."}
  Réponse : {"id":42,"title":"Angular 19","description":"Retour d'expérience...","theme":{"id":1,"title":"Angular"},"user":{"id":3,"name":"Lucas"}}

Contraintes principales :
- email unique côté User ;
- unicité du couple (user_id, theme_id) dans Subscription ;
- mot de passe validé côté DTO (au moins 8 caractères, une majuscule, un chiffre et un caractère spécial) ;
- champs title / description bornés par taille.

2.3.1 Détail des endpoints API implémentés

Le tableau ci-dessous reprend les endpoints observés dans les contrôleurs Spring Boot, avec leur mode d'authentification, les entrées attendues et la nature des réponses retournées.

Endpoint	Méthode	Auth.	Description	Paramètres / body	Réponse / statuts
/api/auth/register	POST	Non	Inscription d'un nouvel utilisateur.	Body : email, name, password	201 Created ; AuthResponse {token}
/api/auth/login	POST	Non	Connexion d'un utilisateur existant.	Body : email, password	200 OK ; AuthResponse {token}
/api/articles	GET	Oui	Liste les articles triés par date de création.	Query : sort=asc|desc	200 OK ; List<ArticleLightDto>
/api/articles/{id}	GET	Oui	Retourne le détail complet d'un article.	Path : id	200 OK ; ArticleDto
/api/articles	POST	Oui	Crée un article pour l'utilisateur connecté.	Body : title, description, themeId	201 Created ; ArticleDto
/api/articles/{id}/comments	POST	Oui	Ajoute un commentaire sur un article.	Path : id ; Body : description	201 Created ; CommentDto
/api/themes	GET	Oui	Liste les thèmes et l'état d'abonnement.	—	200 OK ; List<ThemeDto>
/api/themes/{id}/subscribe	POST	Oui	Abonne l'utilisateur à un thème.	Path : id	204 No Content
/api/themes/{id}/subscribe	DELETE	Oui	Désabonne l'utilisateur d'un thème.	Path : id	204 No Content
/api/me	GET	Oui	Retourne le profil de l'utilisateur connecté.	—	200 OK ; UserDto
/api/me	PUT	Oui	Met à jour le profil connecté.	Body : email, name, password	200 OK ; UserDto
/api/me/subscriptions	GET	Oui	Retourne les abonnements du profil connecté.	—	200 OK ; List<SubscriptionDto>

Points de validation transverses : les routes hors /api/auth/** sont sécurisées par JWT ; les DTO utilisent Jakarta Validation ; le mot de passe doit comporter au moins 8 caractères, une majuscule, un chiffre et un caractère spécial ; l'email utilisateur est unique.

Exemples utiles : POST /api/auth/login avec {"email":"user@example.com","password":"Password1!"} retourne {"token":"<jwt>"}. POST /api/articles avec {"themeId":1,"title":"Angular 19","description":"Retour d'expérience..."} retourne l'article créé.

3. Tests, performance et qualité

3.1 Stratégie de test

Le dépôt contient trois niveaux de tests : tests unitaires front via Jest, tests unitaires et d'intégration back via Spring Boot Test / JUnit, ainsi que des scénarios end-to-end via Cypress. Les rapports générés permettent d'objectiver la couverture réelle observée dans l'archive.

Type de test	Outil / framework	Portée	Résultats
Test unitaire front-end	Jest	Composants, guards, interceptor et services Angular.	15 fichiers de tests, environ 51 cas. Couverture front observée : 81,2 % statements, 71,42 % branches, 76,4 % functions, 80,84 % lines.
Test unitaire back-end	JUnit 5 + Spring Test	Services, mappers, sécurité et gestionnaire d'exceptions.	59 tests backend exécutés, 0 échec, 0 erreur. Couverture JaCoCo : 94,3 % lignes, 75,8 % branches, 92,0 % instructions.
Test d'intégration API	Spring Boot Test	Contrôleurs Auth, Article et Me avec chargement du contexte et vérification des réponses.	Présence de tests d'intégration sur les endpoints critiques d'authentification et de consultation.
Test e2e	Cypress	Parcours utilisateur : authentification, navigation, détail article, profil et thèmes.	3 fichiers E2E, environ 14 scénarios. Les parcours principaux du front sont couverts.

3.2 Rapport de performance et optimisation

Lighthouse et Sonarqube ont été utilisés pour mesurer respectivement la qualité du front et du back.
Optimisations:
- utilisation de DTO légers pour la liste des articles afin de limiter le volume de données transférées ;
- tri délégué à la base de données via Spring Data plutôt que côté client ;
- architecture stateless JWT, évitant une gestion de session serveur lourde ;
- séparation claire des appels API et réutilisation des services Angular ;
- couverture de tests suffisante (70%).

Points d'amélioration identifiés :

- Réduire le code javascript et le css inutilisés

-Améliorer le chargement des images

-Tâches longues sur le thread principal

3.3 Revue technique

Le projet adopte une organisation full-stack avec un front-end développé en Angular 19 et un back-end développé en Spring Boot 3.5.5, en s'appuyant sur une base de données relationnelle MySql. Ce choix technologique permet de bien distinguer les responsabilités entre l'interface utilisateur, la logique métier et la persistance des données.

La partie back-end est organisée en plusieurs couches distinctes : controllers, services, repositories, models, DTO et mappers. Cette séparation favorise la maintenabilité du code ainsi qu'une meilleure compréhension de l'application. L'usage de DTO permet notamment de maîtriser les données exposées par l'API, tandis que l'utilisation de Mapper manuel facilite la transformation entre entités métier et objets échangés avec le front-end.

La partie front-end est organisée autour de pages, services, guards, models et composants partagés, ce qui permet d'isoler les responsabilités liées à l'affichage, à la navigation et aux appels API. Le recours aux standalone components d'Angular ainsi qu'aux Reactive Forms constitue un choix moderne et pertinent. La présence d'un interceptor JWT et de guards d'authentification renforce également la cohérence entre le fonctionnement du front-end et la sécurisation du back-end.

Sur le plan de la sécurité, l'application met en œuvre une authentification basée sur JWT avec Spring Security. Le fonctionnement est de type stateless, ce qui est particulièrement adapté à une API REST. Les routes publiques sont limitées aux endpoints d'authentification, tandis que les autres routes nécessitent un jeton valide. Le mot de passe est chiffré à l'aide de BCrypt, ce qui constitue une bonne pratique essentielle. La présence d'un filtre JWT dédié, d'un service de gestion des tokens et d'une configuration de sécurité centralisée montre que la sécurisation du projet a été pensée de manière structurée.

Le projet intègre une configuration OpenAPI / Swagger, ce qui facilite la compréhension, les tests et la maintenance des endpoints. Cet élément renforce la qualité technique globale du projet, car il améliore la lisibilité du contrat d'API et permet une meilleure communication autour des fonctionnalités exposées.

La gestion des erreurs est globalement satisfaisante. La présence d'un GlobalExceptionHandler permet d'uniformiser les réponses renvoyées par le serveur en cas d'erreur de validation, d'erreur métier ou d'exception plus générale.

Côté back-end, le projet contient de nombreux tests unitaires et d'intégration portant sur les contrôleurs, les services, les mappers, la sécurité et la gestion des exceptions. Côté front-end, des tests sont également présents avec Jest, et une couverture est générée. Le projet intègre aussi Cypress pour les tests end-to-end.

La revue technique met toutefois en évidence plusieurs points d'amélioration. Tout d'abord, certaines fonctionnalités pourraient être rendues plus robustes ou plus évolutives. Par exemple, la liste des articles repose actuellement sur un tri simple, sans véritable système de pagination. Ce fonctionnement reste acceptable pour un volume de données limité, mais il pourrait devenir moins performant si le nombre d'articles augmentait. L'ajout d'une pagination côté serveur constituerait donc une amélioration importante pour la scalabilité de l'application.

De la même manière, des fonctionnalités complémentaires de sécurité, comme la confirmation d'adresse e-mail, la gestion du mot de passe oublié ou une politique de renouvellement des tokens, pourraient être envisagées pour renforcer encore le projet.

Enfin, l'ajout d'outils de modération, de rôles supplémentaires ou de fonctionnalités de recherche permettrait d'améliorer à la fois l'expérience utilisateur et la gestion de la plateforme.

4. Documentation utilisateur et supervision

4.1 FAQ utilisateur

FAQ utilisateur basée sur les parcours réellement visibles dans l'application.

Q : Comment créer un compte ?
R : Depuis la page d'accueil, cliquez sur "S'inscrire", saisissez votre nom, votre email et un mot de passe conforme, puis validez le formulaire.
Q : Comment me connecter ?
R : Accédez à la page "Login", renseignez votre email et votre mot de passe. Après authentification, vous êtes redirigé vers l'espace connecté.
Q : Comment publier un article ?
R : Ouvrez la page "Créer un article", choisissez un thème, saisissez le titre et le contenu, puis validez. L'article est créé et affiché dans son écran de détail.
Q : Comment m'abonner à un thème ?
R : Depuis la page "Themes", cliquez sur le bouton d'abonnement du thème souhaité. Le thème apparaîtra ensuite parmi vos abonnements.
Q : Comment modifier mon profil ?
R : Rendez-vous sur la page "Me", modifiez vos informations puis enregistrez. Le mot de passe doit respecter la règle de complexité imposée par l'application.
Q : Que faire en cas d'erreur 401 ou de déconnexion ?
R : Si le token n'est plus valide, l'application vous redirige vers la page de connexion. Reconnectez-vous pour poursuivre.

4.2 Supervision et tâches déléguées à l'IA

Tâche déléguée	Outil / collaborateur	Objectif	Vérification effectuée
Structuration du rapport et reformulation	ChatGPT	Accélérer la mise en forme d'une documentation technique cohérente.	Relecture manuelle.
Retouche CSS	ChatGPT / Copilot	Gagner du temps et améliorer la propreté du code.	Relecture manuelle et vérification du résultat.
Tests et corrections de bugs	ChatGPT / Copilot	Aider au diagnostic et à l'augmentation de la couverture.	Relecture manuelle et vérification du résultat.
Relecture du code	ChatGPT / Copilot	Vérifier le respect des conventions de nommage et la propreté du code.	Relecture manuelle du code et tests de non-régression.

Tâche déléguée	Outil / collaborateur	Objectif	Vérification effectuée
Explications de certains concepts	ChatGPT	Mieux comprendre des notions techniques plus complexes.	Comparaison avec la documentation et des sources techniques de référence.
Ajout de données dans le script SQL	ChatGPT	Accélérer la création de données de test.	Relecture manuelle.

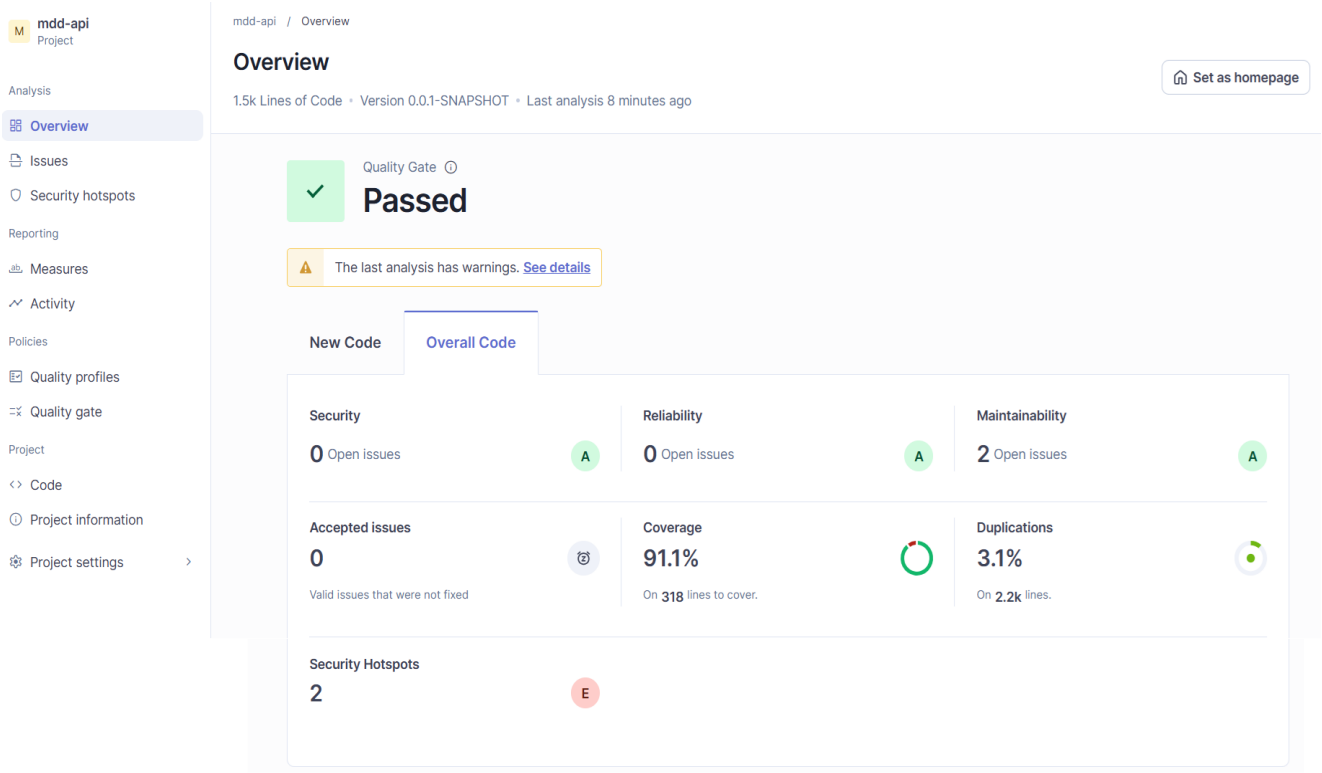
5. Annexes

- Rapport de couverture backend JaCoCo : back/target/site/jacoco/index.html
- Rapports de tests backend : back/target/surefire-reports/
- Couverture front : front/coverage/cypress/index.html
- Scénarios end-to-end : front/cypress/e2e/
- Ressources UI : front/src/assets/
- Captures ajoutées ci-dessous : qualité logicielle, performance et parcours UI desktop/mobile.

Toutes les captures sont également en dehors du PDF pour plus de lisibilité.

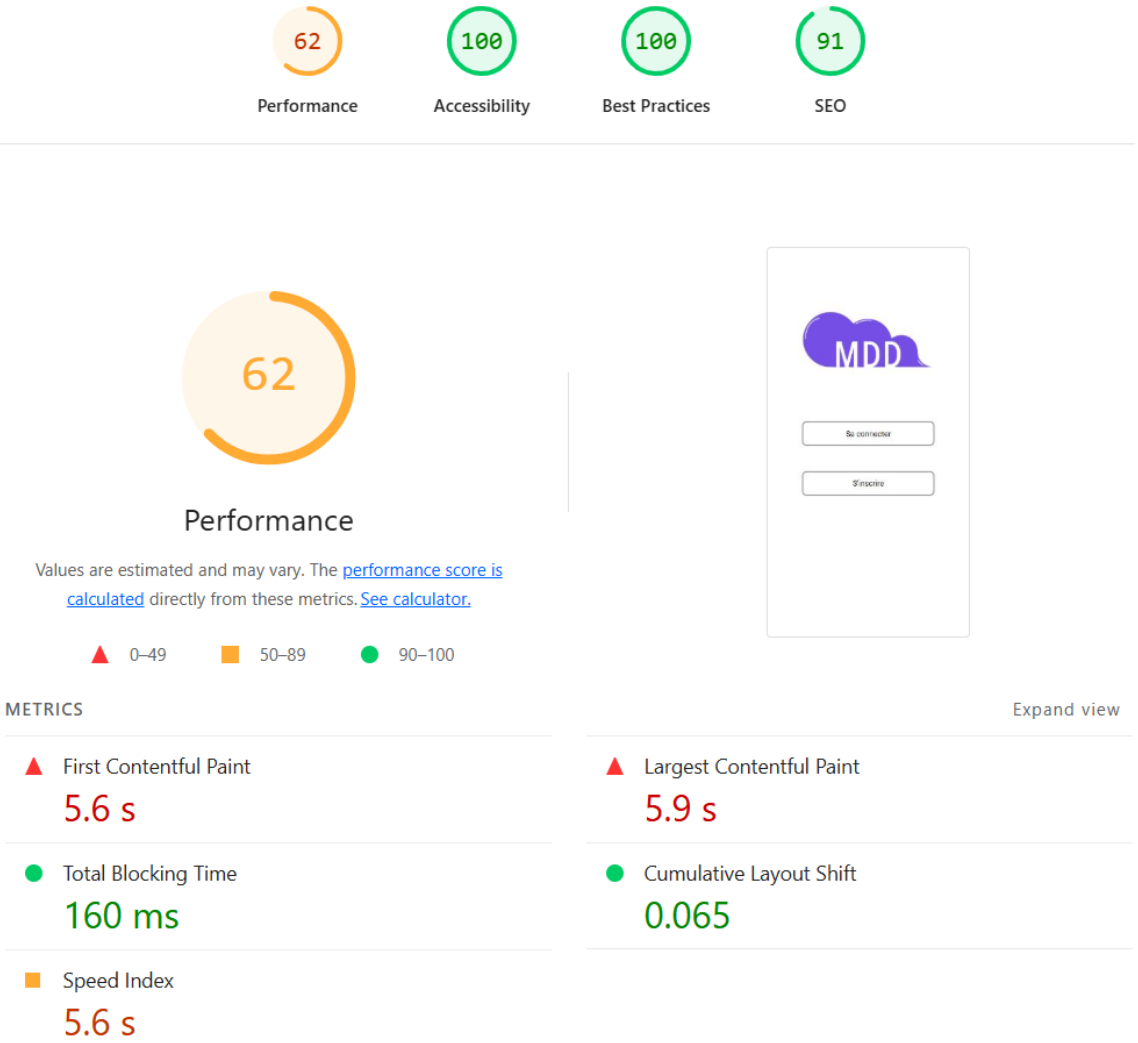
5.1 Captures et rapports complémentaires

Annexe A.1 - SonarQube



Sonar.png

Annexe A.2 - Lighthouse



lighthouse.png

Annexe A.3 - JaCoCo

mdd-api

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.openclassrooms.mddapi.security		87 %		61 %	15	43	14	102	6	30	0	5
com.openclassrooms.mddapi.service		92 %		72 %	11	35	0	114	6	26	0	4
com.openclassrooms.mddapi.controller		89 %		n/a	2	16	2	27	2	16	0	4
com.openclassrooms.mddapi		37 %		n/a	1	2	2	3	1	2	0	1
com.openclassrooms.mddapi.mapper		100 %		100 %	0	21	0	54	0	13	0	5
com.openclassrooms.mddapi.exception		100 %		100 %	0	7	0	15	0	6	0	1
com.openclassrooms.mddapi.config		100 %		n/a	0	1	0	1	0	1	0	1
Total	100 of 1 250	92 %	15 of 62	75 %	29	125	18	316	15	94	0	21

Jacoco.png

Annexe A.4 - Javadoc

Contrôleur REST de gestion des articles et des commentaires associés.

Constructor Summary

Constructors

Constructor	Description
`ArticleController(ArticleService articleService)`	Construit le contrôleur des articles.

Method Summary

All Methods | **Instance Methods** | **Concrete Methods**

Modifier and Type	Method	Description
`CommentDto`	`addComment(UserPrincipal principal, Integer id, @Valid CreateCommentRequest request)`	Ajoute un commentaire sur un article existant.
`ArticleDto`	`create(UserPrincipal principal, @Valid CreateArticleRequest request)`	Crée un nouvel article pour l'utilisateur connecté.
`ArticleDto`	`get(Integer id)`	Retourne le détail d'un article.
`List<ArticleLightDto>`	`list(String sort)`	Retourne la liste des articles triés par date de création.

Javadoc.png

Annexe A.5 - Jest

All files

92.73% Statements 370/399 **73.07%** Branches 19/26 **78.37%** Functions 58/74 **92.81%** Lines 336/362

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
app	100%	6/6	100%	0/0	100%	0/0	100%	4/4
app/components/header	100%	13/13	100%	0/0	100%	2/2	100%	12/12
app/core	100%	19/19	100%	4/4	100%	3/3	100%	17/17
app/guards	100%	20/20	100%	2/2	100%	2/2	100%	18/18
app/pages/article-create	93.33%	42/45	50%	1/2	85.71%	6/7	95.23%	40/42
app/pages/article-detail	93.61%	44/47	66.66%	4/6	85.71%	6/7	95.45%	42/44
app/pages/articles	100%	25/25	50%	2/4	100%	6/6	100%	24/24
app/pages/home	100%	5/5	100%	0/0	100%	0/0	100%	3/3
app/pages/login	100%	30/30	100%	1/1	100%	3/3	100%	28/28
app/pages/me	93.1%	54/58	50%	1/2	78.57%	11/14	94.44%	51/54
app/pages/register	100%	31/31	100%	1/1	100%	3/3	100%	29/29
app/pages/themes	93.1%	27/29	100%	1/1	85.71%	6/7	92.3%	24/26
app/services	76.05%	54/71	66.66%	2/3	50%	10/20	72.13%	44/61

Jest.png

Annexe A.6 - Cypress

All files

81.2% Statements 229/282 **71.42%** Branches 40/56 **76.4%** Functions 68/89 **80.84%** Lines 211/261

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

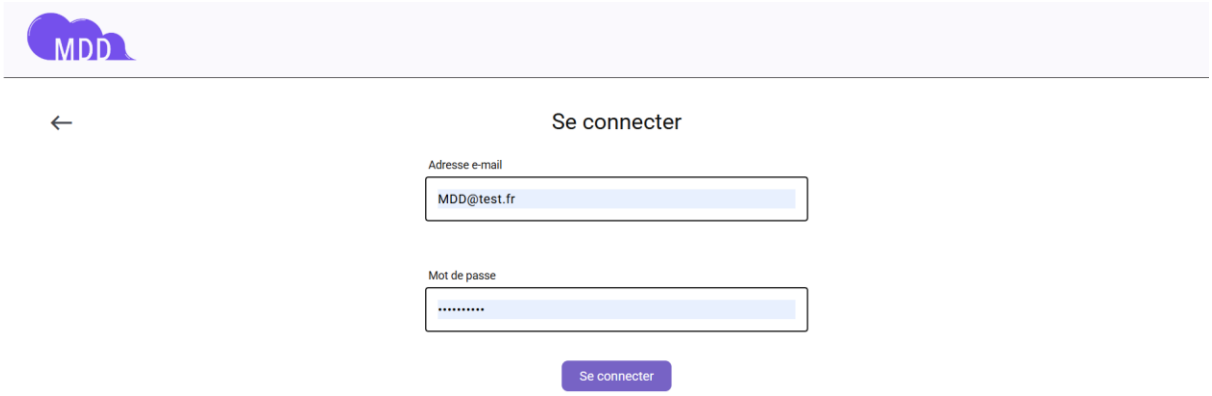
File	Statements		Branches		Functions		Lines	
app	100%	5/5	100%	2/2	100%	1/1	100%	4/4
app/components/header	66.66%	4/6	100%	2/2	66.66%	2/3	60%	3/5
app/core	84.61%	11/13	75%	3/4	100%	3/3	83.33%	10/12
app/environments	100%	1/1	100%	0/0	100%	0/0	100%	1/1
app/guards	91.66%	11/12	100%	2/2	100%	2/2	91.66%	11/12
app/pages/article-create	78.12%	25/32	50%	2/4	75%	6/8	80%	24/30
app/pages/article-detail	91.42%	32/35	87.5%	7/8	87.5%	7/8	90.9%	30/33
app/pages/articles	72.22%	13/18	33.33%	2/6	71.42%	5/7	70.58%	12/17
app/pages/home	100%	2/2	0%	0/2	100%	1/1	100%	1/1
app/pages/login	89.47%	17/19	100%	3/3	75%	3/4	88.88%	16/18
app/pages/me	82.97%	39/47	50%	2/4	73.33%	11/15	84.09%	37/44
app/pages/register	65%	13/20	100%	3/3	50%	2/4	63.15%	12/19
app/pages/themes	77.27%	17/22	66.66%	2/3	75%	6/8	80%	16/20
app/services	78%	39/50	76.92%	10/13	76%	19/25	75.55%	34/45

Cypress.png

Annexe B.1 - Interface desktop 1

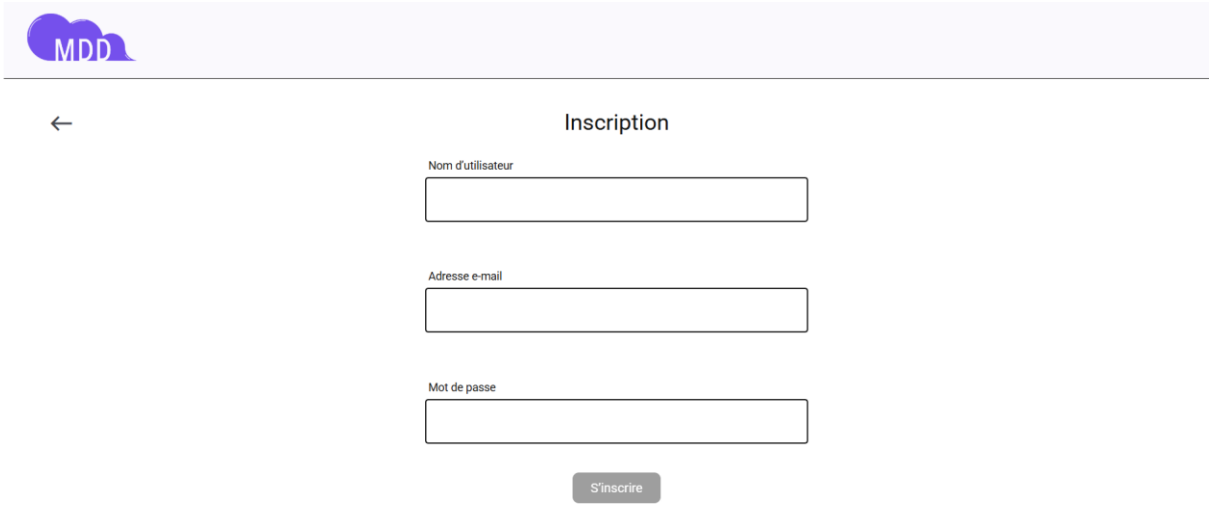
1.png

Annexe B.2 - Interface desktop 2



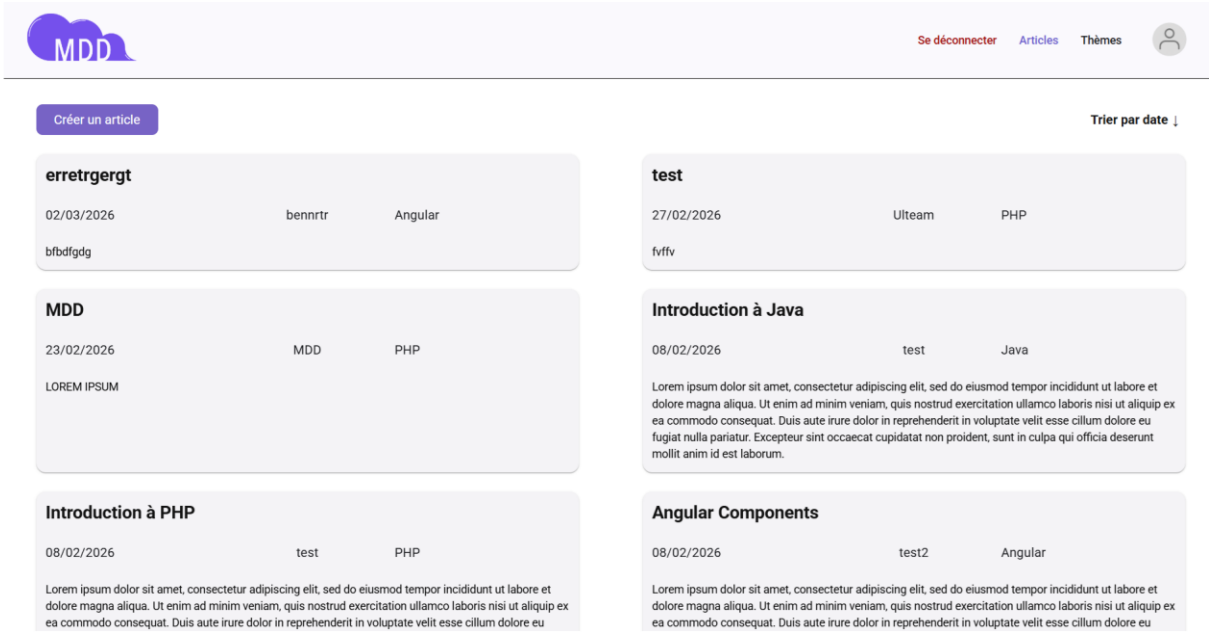
2.png

Annexe B.3 - Interface desktop 3



3.png

Annexe B.4 - Interface desktop 4

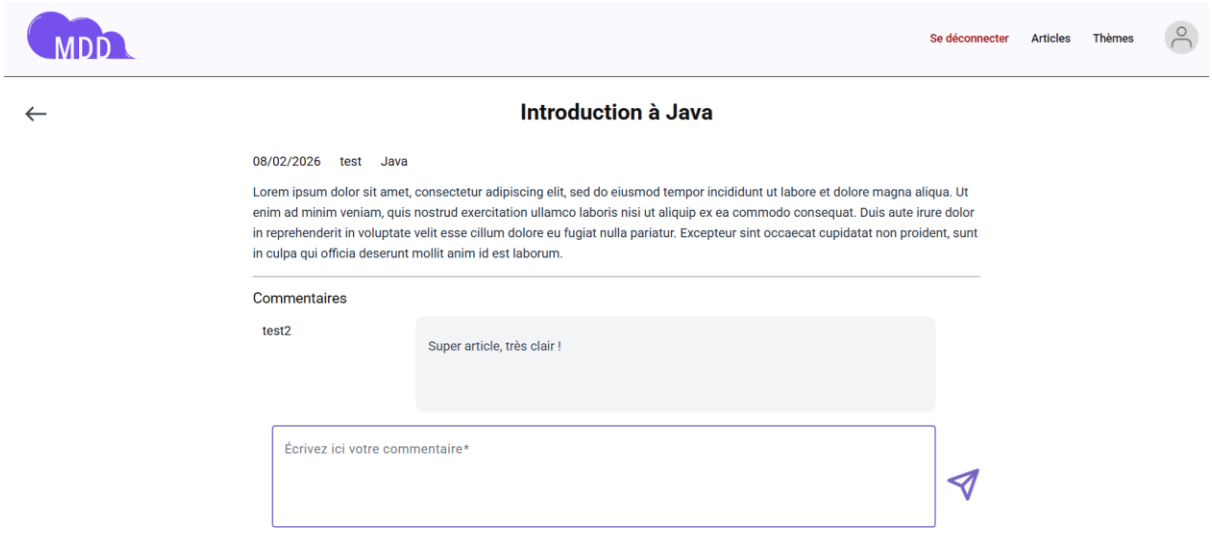


4.png

Annexe B.5 - Interface desktop 5

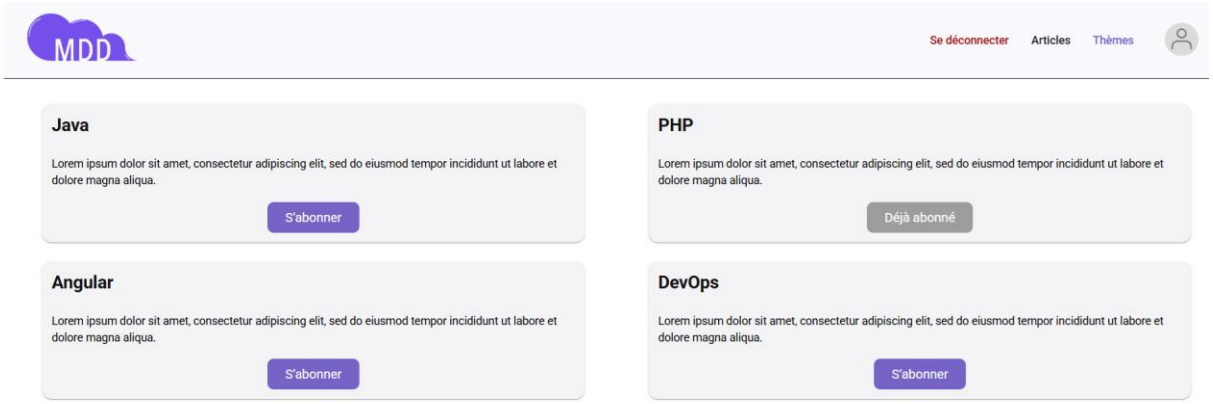
5.png

Annexe B.6 - Interface desktop 6



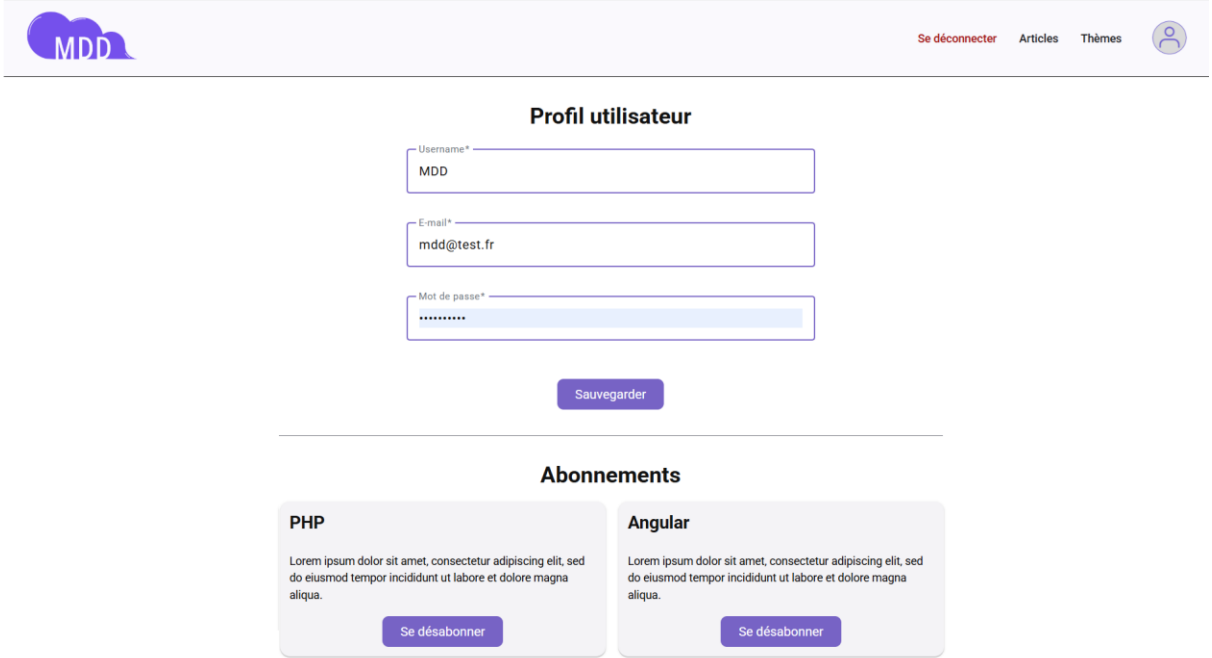
6.png

Annexe B.7 - Interface desktop 7



7.png

Annexe B.8 - Interface desktop 8

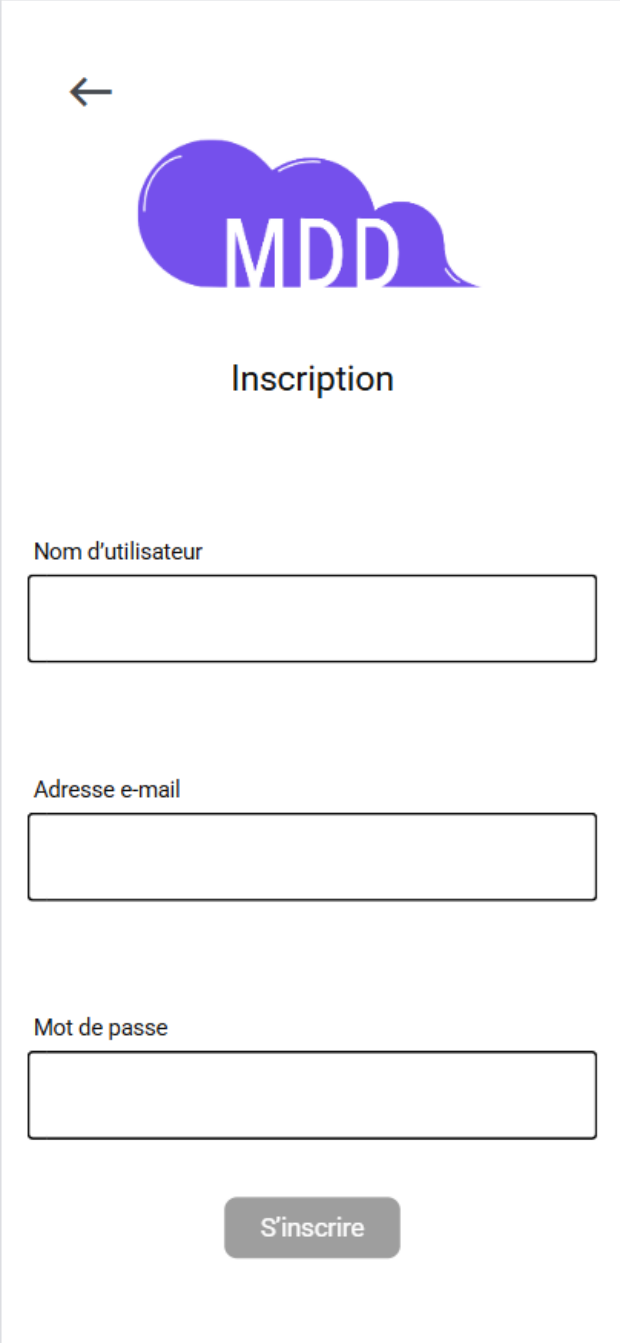


8.png

Annexe C.1 - Interface mobile 1

1.png

Annexe C.2 - Interface mobile 2



2.png

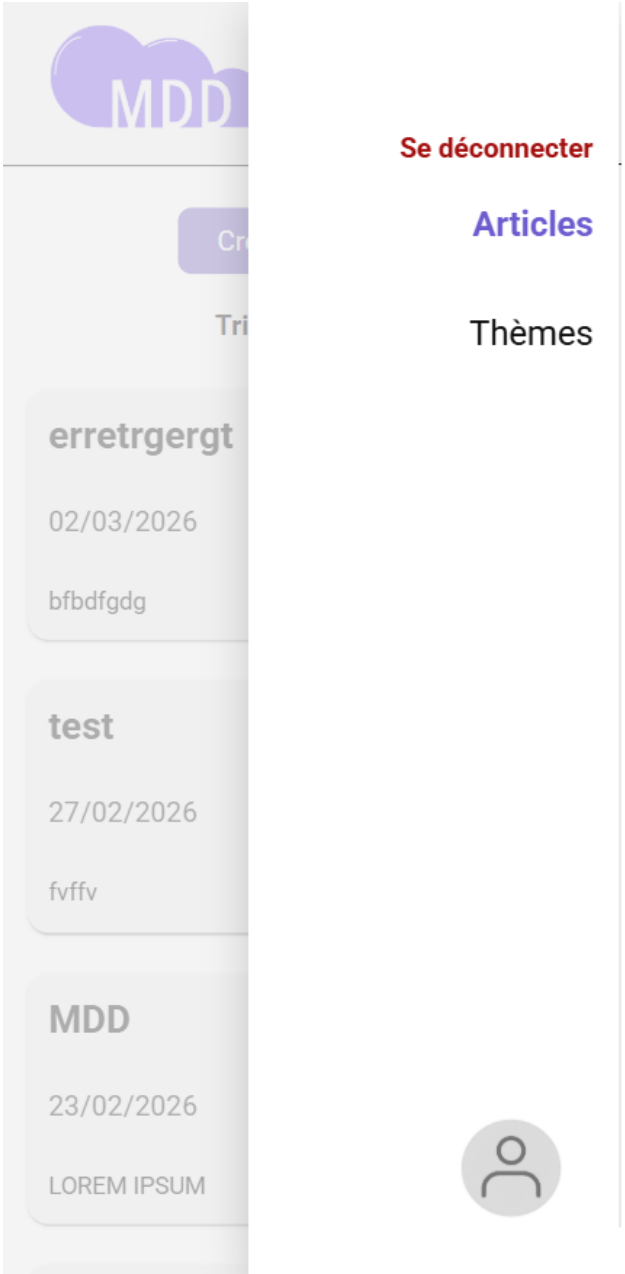
Annexe C.3 - Interface mobile 3

3.png

Annexe C.4 - Interface mobile 4

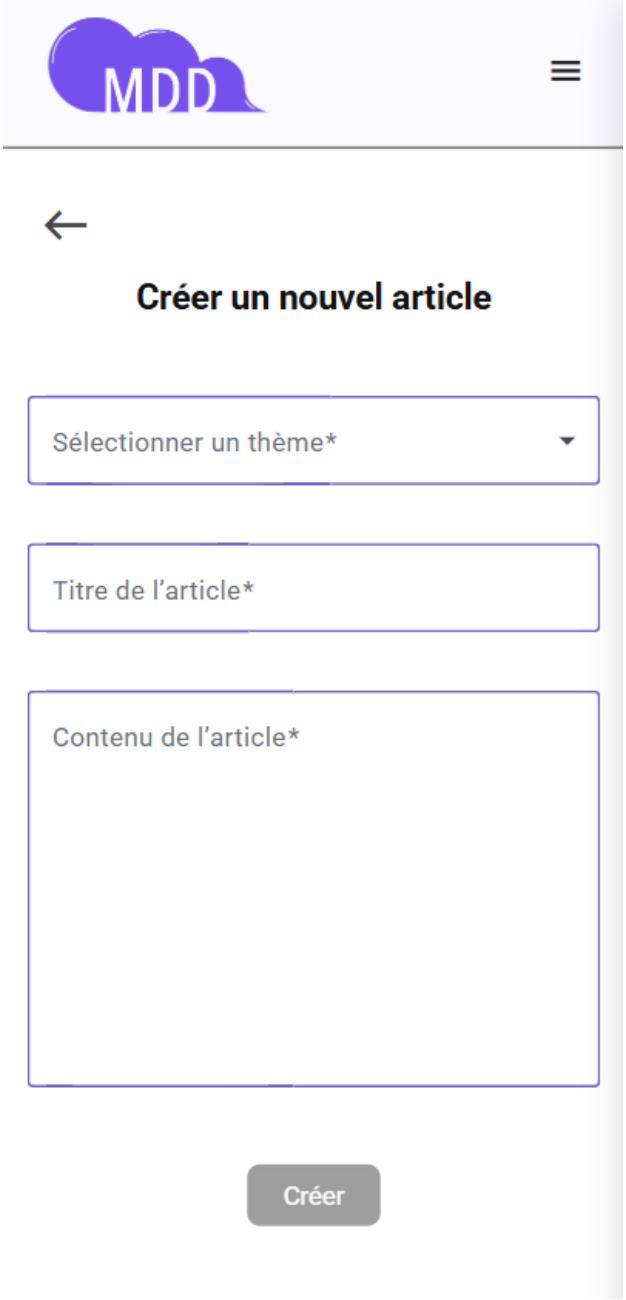
4.png

Annexe C.5 - Interface mobile 5



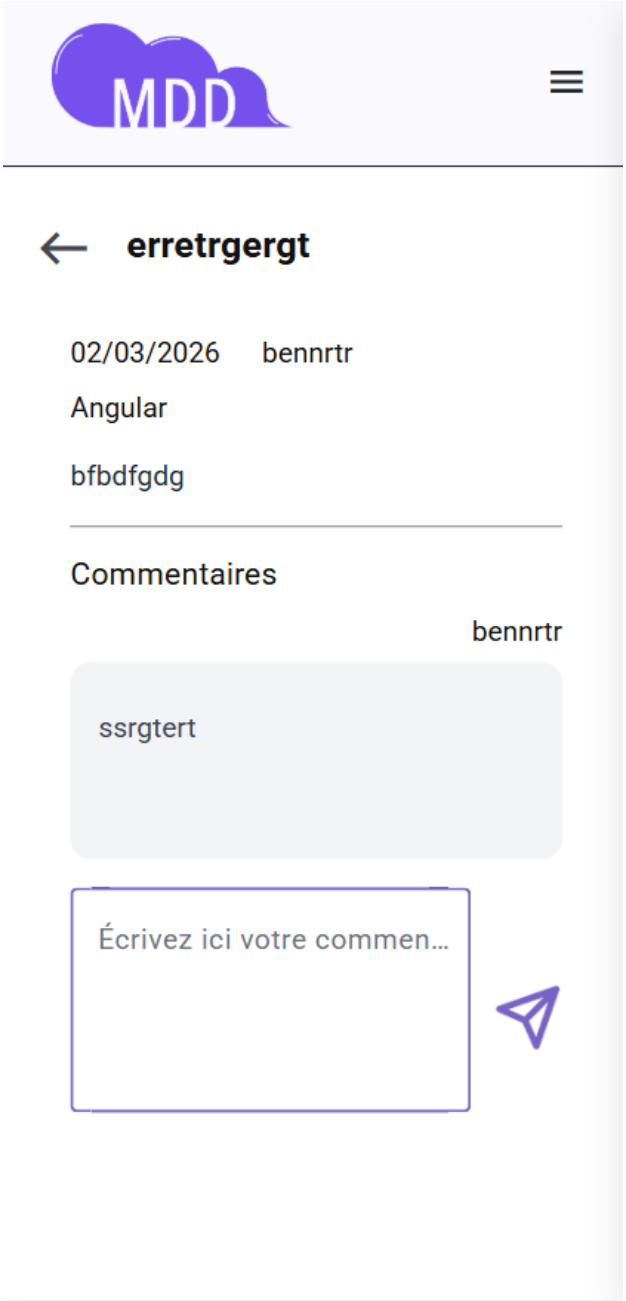
5.png

Annexe C.6 - Interface mobile 6



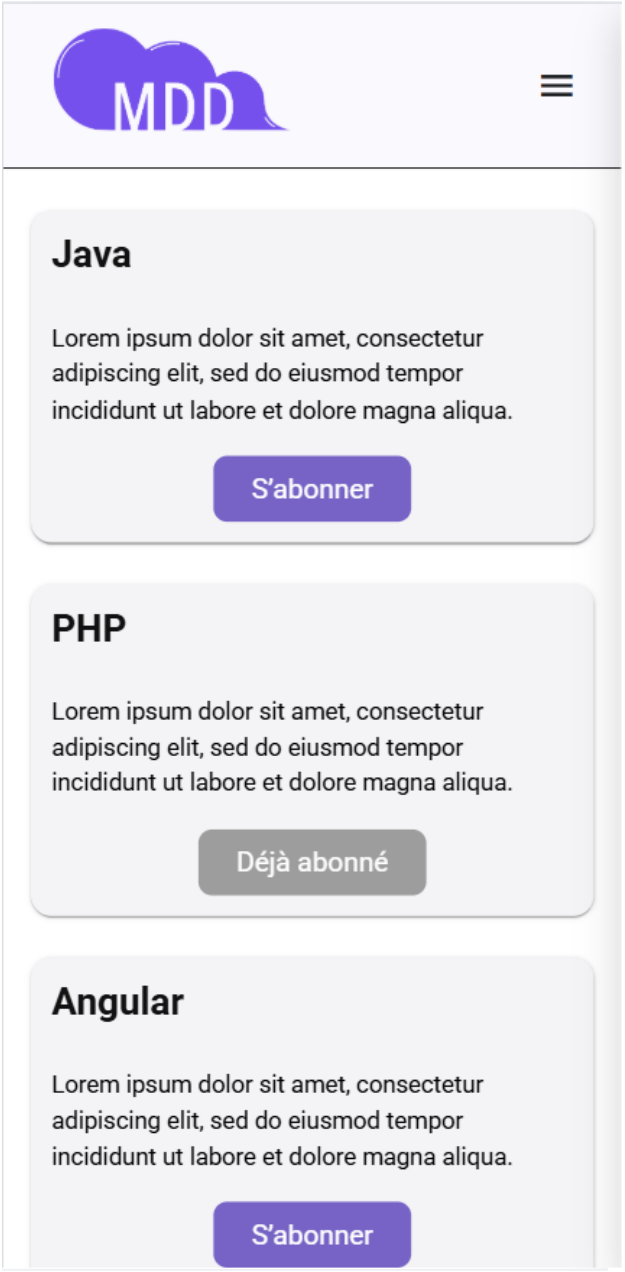
6.png

Annexe C.7 - Interface mobile 7



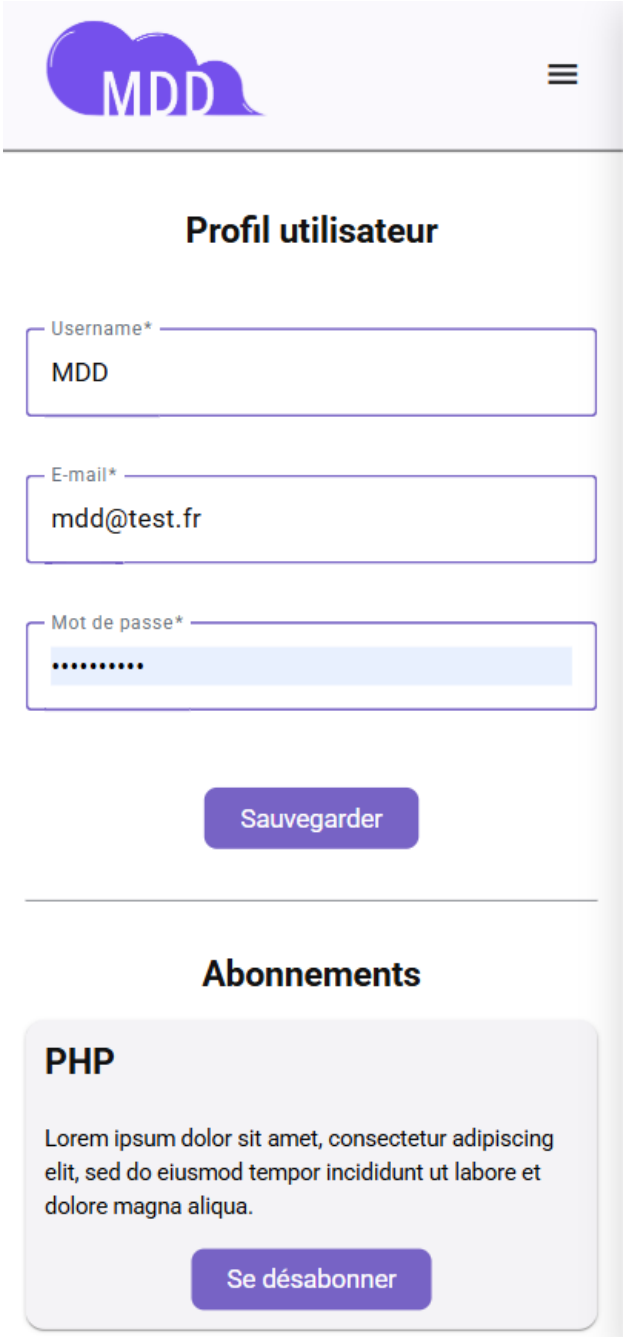
7.png

Annexe C.8 - Interface mobile 8



8.png

Annexe C.9 - Interface mobile 9



9.png